

Building MindCache: Designing an Agentic Memory System for Long-Term AI

20 July 2026 06:08

How a simple attempt to make an AI remember gradually became an exploration of what long-term memory for AI agents should actually look like.

Part I — It Started with a Much Smaller Problem

When people talk about AI memory, the conversation usually begins with a simple question:

How do we give an AI long-term memory?

At first, it sounds like a straightforward engineering problem. Store important conversations, retrieve them when needed, and provide them back to the model. The AI remembers what it would otherwise forget.

For a while, I believed that too.

This project didn't begin as a research project. It wasn't inspired by a benchmark, nor was it an attempt to design a new memory architecture. It started with something much simpler.

Frustration.

Like many developers, I use ChatGPT every day. Over time, it learned pieces of my workflow—the projects I was building, the technologies I preferred, design decisions we'd already discussed, and long-term goals that shaped almost every technical conversation.

Then, seemingly at random, some of that context would disappear.

I'd find myself explaining the same preference again. Revisiting decisions that had already been made. Reconstructing context that had existed only days earlier.

None of these were major failures. They were small interruptions that gradually accumulated into friction. Every forgotten preference meant another explanation. Every lost design decision meant rebuilding context that already existed. Individually, they were minor inconveniences. Collectively, they made conversations feel surprisingly fragile.

Eventually, I found myself asking a simple question.

What if I built a memory system that could actually remember?

That was the entire goal.

I wasn't thinking about agentic memory, hierarchical knowledge graphs, retrieval pipelines, or long-term reasoning. I simply wanted a reliable memory layer that an AI assistant could depend on instead of repeatedly asking for information it had already learned.

I assumed someone had already solved this problem.

The more I read, however, the more I realized I didn't actually understand what *memory* meant in the context of AI.

- What qualifies as a memory?
- Should every conversation become one?
- Should memories be stored exactly as they happened, or distilled into structured knowledge?
- How should user preferences be represented?
- What happens when those preferences change?
- Are long-term decisions fundamentally different from factual knowledge?

I didn't have good answers.

Whenever I encounter a system I don't fully understand, I tend to go back to first principles. Rather than treating existing systems as black boxes, I started reading papers, studying architectures, comparing design choices, and asking why different systems had been built the way they were.

Initially, I wasn't trying to build something better.

I was trying to understand why existing systems worked—and, more importantly, where they didn't.

I expected to come away with implementation ideas.

Instead, I came away with better questions.

As I experimented with different approaches, I kept encountering the same pattern. Existing systems were remarkably good at remembering isolated facts, but they became increasingly fragile as conversations stretched over weeks or months. Preferences evolved. Decisions changed. Projects branched in new directions. These weren't failures that appeared in a short demo, but they became obvious during long-running interactions.

Around that time, I came across the **BEAM** benchmark.

What caught my attention wasn't the leaderboard or the proposed methods. It was the questions the benchmark was asking.

Instead of evaluating whether a system could recall a single fact, BEAM asked whether an agent could remain consistent over long periods of interaction.

Could it remember preferences that evolved over time?

Could it reason about decisions made weeks earlier?

Could it connect pieces of information that weren't obviously related, but were still essential to answering a question correctly?

Those were exactly the kinds of failures I had already been observing.

BEAM didn't change the direction of the project overnight. Instead, it gave me a vocabulary for observations I hadn't yet fully articulated. It confirmed that the challenge wasn't simply storing more memories. The harder problem was understanding **what memories are, how they evolve, and how they should influence future reasoning**.

Without realizing it, the project quietly changed direction.

It stopped being an attempt to build a better personal memory layer.

It became an engineering investigation into a much larger question.

How should an AI remember?

That question eventually led through months of experiments, architectural redesigns, benchmark evaluations, failed assumptions, and countless engineering trade-offs.

Before I could design a better memory architecture, though, I first had to understand why existing memory systems failed.

That turned out to be much harder than building one from scratch.

MindCache is open source.

- **GitHub:** [faisalhussain-devs/MindCache](https://github.com/faisalhussain-devs/MindCache)
- **PyPI:** `pip install mindcache`

Part II — Understanding Why Memory Systems Fail

When I first started studying long-term memory systems, I expected to find competing implementations.

Instead, I found competing definitions of what *memory* actually meant.

Projects such as **Mem0**, **MemGPT**, **Zep**, **LangMem**, and many others are genuinely impressive. They dramatically improve conversational continuity by extracting useful information, storing it efficiently, and retrieving it when needed. For the problems they were designed to solve—persistent user preferences and factual recall—they perform remarkably well.

Initially, I assumed the remaining work was mostly engineering.

Better embeddings.

Better rerankers.

Faster databases.

Smarter retrieval pipelines.

That assumption didn't last long.

As I read more papers and evaluated existing systems, I noticed something interesting. The most valuable papers weren't necessarily the ones proposing new architectures—they were the ones exposing new failure modes.

The BEAM benchmark was particularly interesting, not because it was harder than previous benchmarks, but because it asked fundamentally different questions. Instead of testing whether a system could recall an isolated fact, it evaluated whether an agent could reason over knowledge accumulated across weeks or months of interaction.

Could it remain consistent as preferences evolved?

Could it reason about earlier decisions?

Could it reconstruct a user's history rather than simply retrieve individual memories?

Those were exactly the kinds of failures I had already started encountering in my own experiments.

Over the next few months, a pattern gradually emerged. The failures looked unrelated at first—a shopping recommendation here, a changing preference there, a broad summarization query somewhere else—but they all pointed toward the same conclusion.

The challenge wasn't simply retrieving memories.

It was understanding **which memories mattered, how they related to one another, and how they should influence future reasoning**.

Failure 1 — When Retrieval Is Technically Correct but Practically Wrong

The first failure almost fooled me because, on paper, nothing was actually broken.

Imagine asking an AI assistant:

"I'm planning to buy a mattress. Which one should I choose?"

A modern retrieval pipeline behaves exactly as expected. It retrieves previous conversations about mattress brands, firmness, comfort, pricing, reviews, and buying recommendations.

Everything looks correct.

The embeddings work.

The reranker works.

The retrieval metrics look excellent.

Yet the final answer can still be fundamentally wrong.

Months earlier, buried inside an unrelated conversation, the user had mentioned:

"Whenever I buy something expensive, the first thing I care about is the warranty."

That memory contains almost nothing semantically related to mattresses. A vector retriever is unlikely to rank it highly, and keyword search will almost certainly miss it.

Yet it is arguably the single most important memory for answering the question.

The assistant shouldn't simply recommend the most comfortable mattress.

It should recommend the one with the best warranty.
That example fundamentally changed how I thought about retrieval.
I had been optimizing for **similarity**.
The user cared about **utility**.
Those are very different objectives.
Retrieval wasn't failing because it couldn't find relevant memories.
It was failing because relevance alone wasn't enough.

Failure 2 — Memory Without Time Isn't Memory

The second failure looked completely different.

Suppose a user says:

"I'm learning TensorFlow."

Months later:

"I've switched to PyTorch."

Later still:

"From now on, use PyTorch for all code examples."

Every one of those memories still exists.

Nothing has been forgotten.

The problem is that memory isn't simply a collection of facts.

It's a timeline.

Retrieval now has to determine not only **what happened**, but **when it happened**, and whether newer information supersedes older information.

The same pattern appears everywhere.

Projects begin and are abandoned.

Goals evolve.

Recommendations become outdated.

Preferences change.

Chronology isn't merely metadata.

It's part of the memory itself.

This also changed how I thought about retrieval. I had assumed retrieval's job was to recover memories. Increasingly, it seemed that its real job was to gather the **evidence** needed to answer a question correctly.

Those aren't the same thing.

Sometimes the most important evidence isn't the most similar memory—or even one that appears to be about the same topic.

Failure 3 — Broad Questions Break Local Retrieval

The third failure only became obvious once I started asking broader questions.

Questions such as:

"Tell me about my machine learning journey."

or

"Summarize everything we've learned while building this project."

These aren't asking for a single fact.

They're asking for a story.

There is no single keyword, embedding, or memory that contains the answer.

A good response might require memories about books, experiments, architectural decisions, benchmark results, failures, future plans, and months of incremental learning.

Traditional retrieval systems excel at finding the **most relevant** memories.

Broad questions require something different.

They require retrieving **enough diverse evidence** to build a coherent picture.

At that point, I stopped thinking of broad retrieval as a search problem.

It had become a **coverage problem**.

Eventually, I would come to think of it as an **evidence planning problem**.

Failure 4 — People Change. Memories Don't.

Another failure emerged as conversations accumulated.

People contradict themselves.

Not because earlier memories were wrong.

Because people change.

A preference recorded six months ago may no longer be true.

A recommendation that once made sense may become obsolete.

A project can pivot several times before finally stabilizing.
Treating every memory as an immutable fact creates a different kind of failure.
The system faithfully remembers everything.
But it no longer understands what is still true.
Memory isn't static.
It has a lifecycle.

Failure 5 — Facts Are Not Decisions

The final realization surprised me the most.
Facts and decisions behave fundamentally differently.
Facts are usually independent.
Learning Java doesn't invalidate knowing Python.
Decisions don't work that way.
They evolve.
An idea is proposed.
Modified.
Rejected.
Revisited.
Eventually finalized.
Those aren't separate memories.
They're chapters in the same decision.
Treating each discussion as an isolated memory destroys the reasoning that led to the final outcome.
Long-term agents don't just need to remember **what** was decided.
They need to remember **why**.

By this point, I had stopped thinking about memory systems as searchable databases.
The information required to answer most questions was already there.
The real challenge was deciding **how that information should be represented, organized, and allowed to evolve before retrieval ever began**.
That realization became the turning point of the project.
I stopped searching for a better retrieval pipeline.
Instead, I began thinking about a different kind of memory architecture altogether.
The design principles behind **MindCache** emerged directly from that shift in perspective.

Failure	What Went Wrong	Architectural Response
Retrieval was technically correct but practically wrong	Semantic similarity did not capture the memories that actually influenced the answer.	User Memories preserve long-term preferences and high-impact personal context so retrieval considers utility, not just similarity.
Memory without time	Facts alone could not represent how projects, preferences, and goals evolved.	Episodic Memories preserve chronology, allowing retrieval to reason about sequences of events instead of isolated facts.
Broad questions broke local retrieval	Searching for a handful of similar memories could not provide sufficient coverage for multi-topic reasoning.	Hierarchical Knowledge + Summaries organize related information and enable retrieval to navigate from broad concepts to detailed evidence.
People changed, but memories didn't	Treating every memory as immutable caused outdated preferences and obsolete information to accumulate.	Living Knowledge Base continuously updates, consolidates, and reorganizes memories as conversations evolve.
Facts are not decisions	Independent memories lost the reasoning process behind long-running design and project decisions.	Decision Memories model decisions as evolving entities, preserving both their current state and the reasoning that produced them.

Table 1. From Failure to Architecture. Each failure uncovered during development ultimately motivated a corresponding architectural component in MindCache.

Part III — Designing MindCache

By the time I had finished investigating those failures, something unexpected had happened.

I had stopped looking for a better retrieval algorithm.

Or a better embedding model.

Or a better reranker.

Or a more sophisticated vector database.

Those ideas were still useful, but they no longer felt fundamental.

Every retrieval failure I investigated seemed to point somewhere else—not toward retrieval itself, but toward how the memories had been represented long before retrieval ever began.

My instinct had initially been the same as everyone else's. If retrieval isn't finding the right memories, make retrieval smarter.

Use better embeddings.

Combine vector search with BM25.

Add rerankers.

Increase Top-K.

Expand the context window.

Every one of those improvements helped.

None of them solved the underlying problem.

The more experiments I ran, the more obvious something became.

Retrieval is the most constrained stage of a memory system.

It has only milliseconds to decide what information matters. It cannot reorganize an entire knowledge base, reinterpret months of conversations, resolve contradictory preferences, or reconstruct a user's decision history every time a query arrives.

Retrieval is an online operation.

It must remain fast.

That means almost every expensive reasoning task has to happen **before retrieval ever begins**.

That realization became the foundation of MindCache.

Instead of asking,

How can retrieval become smarter?

I started asking a different question.

What understanding can be computed once during ingestion so retrieval never has to compute it again?

Looking back, almost every component of MindCache emerged from trying to answer that single question.

Principle 1 — Intelligence Belongs During Ingestion

Most memory systems treat ingestion as a preprocessing step.

Extract memories.

Generate embeddings.

Store them.

Move on.

The longer I worked on MindCache, the less convincing that pipeline became.

Ingestion isn't simply where memories are stored.

It's where they are **understood**.

Unlike retrieval, ingestion isn't constrained by user latency. The user never waits for it to finish, giving the system an opportunity to perform expensive reasoning once instead of repeatedly.

During ingestion the system can:

- identify relationships between memories,
- resolve contradictions,
- track evolving decisions,
- organize knowledge into topics,
- generate summaries,
- maintain metadata,
- and restructure its knowledge hierarchy when necessary.

Every expensive operation performed once during ingestion becomes effectively free for every future query.

Rather than storing conversations and hoping retrieval can understand them later, MindCache shifts as much reasoning as possible into ingestion.

Retrieval shouldn't create understanding.

It should exploit understanding that has already been created.

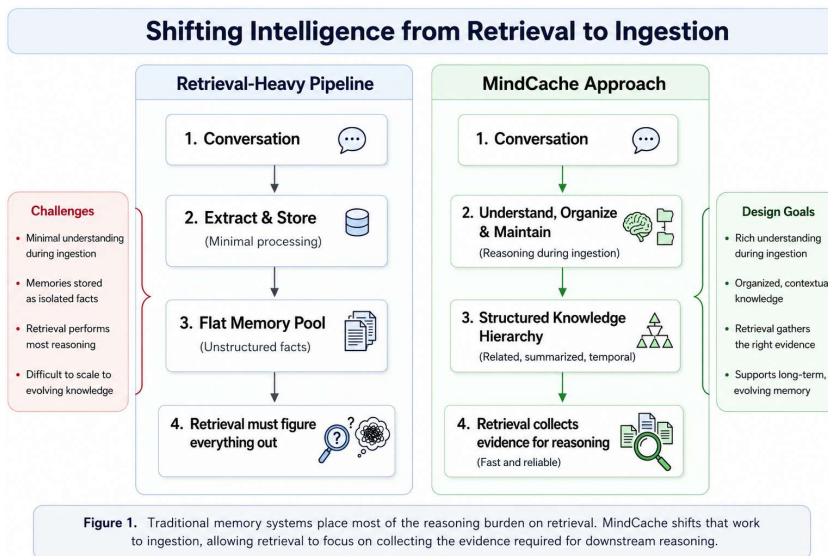


Figure 1.

Principle 2 — Retrieval Retrieves Evidence, Not Memories

One idea kept resurfacing throughout the project.

Retrieval isn't trying to recover memories.

It's trying to assemble the evidence required to answer a question.

That distinction explains many of the failures from the previous chapter.

The mattress example wasn't failing because mattress-related memories were missing.

The most important evidence simply wasn't about mattresses.

Broad questions weren't difficult because embeddings were inaccurate.

They required evidence scattered across many unrelated topics.

Temporal reasoning didn't fail because memories were absent.

It failed because chronology itself was part of the evidence.

Similarity is therefore only one signal.

The real objective is assembling the smallest collection of evidence that allows the language model to reason correctly.

Principle 3 — Organization Is Part of the Memory

I initially believed retrieval quality depended primarily on search algorithms.

Eventually I realized it depended far more on how knowledge had been organized before retrieval began.

A sophisticated reranker cannot compensate for a poorly organized memory store.

Likewise, an embedding cannot provide context that was never represented in the first place.

Human memory isn't simply a collection of isolated experiences.

It groups related ideas.

Builds hierarchies.

Forms abstractions.

Creates relationships between concepts.

Those relationships aren't an optimization layered on top of memory.

They are part of the memory itself.

That realization fundamentally changed how I thought about long-term memory.

Organization wasn't separate from storage.

Organization was storage.

Principle 4 — Different Memories Behave Differently

As the system became more complex, another assumption began to break down.

I had been treating every piece of information as though it behaved the same way.

It didn't.

A factual observation behaves differently from a user preference.

A project decision behaves differently from an episodic event.

Some memories rarely change.

Others evolve continuously.

Some influence almost every future conversation.
Others matter only under very specific circumstances.
Representing every memory in the same way inevitably forced retrieval to make unnecessary compromises.
The problem wasn't retrieval.
It was assuming every memory followed the same lifecycle.

Principle 5 — Memory Is a Living System

One of my earliest assumptions was that memory simply accumulated over time.

Conversations accumulate.

People don't.

Preferences change.

Projects pivot.

Ideas mature.

Decisions are revised.

Knowledge evolves.

If a memory system only knows how to append new information, it gradually becomes less useful—not because it forgets, but because it remembers everything equally.

Long-term memory isn't just about remembering.

It's about maintaining what has already been remembered.

Updating.

Consolidating.

Reorganizing.

Sometimes even recognizing that yesterday's truth is no longer today's.

Memory isn't an archive.

It's a continuously evolving knowledge system.

From Principles to Architecture

None of these principles existed when I started building MindCache.

They weren't designed up front.

They emerged gradually because every architectural experiment kept pointing toward the same conclusion.

The system wasn't failing because it couldn't search.

It was failing because it didn't yet understand what it had stored.

Once that became clear, the architecture almost designed itself.

Knowledge required organization.

Different memories required different representations.

Summaries became hierarchical.

The memory graph needed to evolve alongside the user.

And ingestion transformed from a simple storage pipeline into the component responsible for maintaining the entire knowledge system.

The rest of MindCache's architecture is simply the engineering consequence of these principles.

The next chapter explores how those ideas evolved into the final architecture.

Part IV — The Architecture of MindCache

The design principles from the previous chapter answered **what** a long-term memory system should do.

The next challenge was turning those principles into an architecture that could operate continuously as conversations evolved.

That architecture wasn't designed all at once. Each component emerged to solve a specific limitation discovered during development. Over time, however, those individual solutions converged into a single system with one objective:

Build understanding during ingestion so retrieval can focus on gathering evidence rather than creating it.

The final architecture is organized around three core ideas:

- **A living knowledge hierarchy** that provides structure and context for every memory.
- **Specialized memory representations** that recognize different kinds of information behave differently.
- **A smart ingestion pipeline** that continuously maintains and evolves the knowledge base as new conversations arrive.

Together, these components transform memory from a collection of stored conversations into an evolving knowledge system.

4.1 — Building a Living Knowledge Hierarchy

Most long-term memory systems store memories as independent records.

A conversation ends.

Useful information is extracted.

Embeddings are generated.

The memories are stored.

Whenever a new question arrives, retrieval searches those stored memories and returns the most relevant ones.

This approach works remarkably well when questions depend on a handful of related memories.

It begins to struggle once knowledge becomes larger, more interconnected, and continuously evolving.

The failures described in the previous chapter—broad summarization, temporal reasoning, and fragmented long-running projects—all share the same underlying cause.

The knowledge exists.

It simply lacks structure.

MindCache addresses this by organizing memories into a **living knowledge hierarchy** rather than treating them as isolated documents.

Instead of existing independently, every memory belongs to a topic, and every topic belongs to a broader concept.

For example, a memory about **PyTorch** no longer exists in isolation. It naturally becomes part of a hierarchy such as:

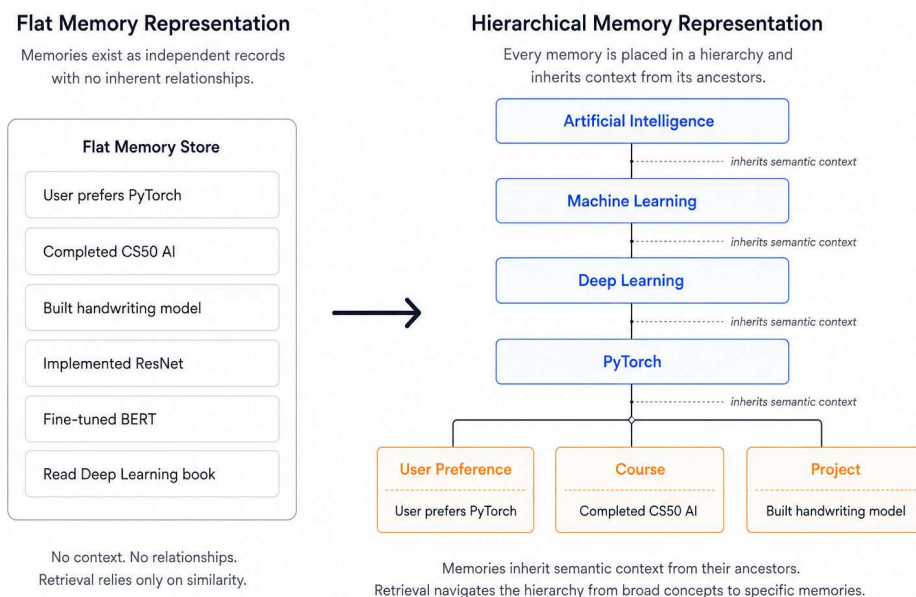


Figure 2. Unlike traditional memory systems that store independent records, MindCache organizes memories within a semantic hierarchy. Each memory inherits context from its ancestors, allowing retrieval to navigate from broad concepts to increasingly specific evidence.

Likewise, a user preference such as

"Whenever I buy something expensive, warranty matters more than anything else."

is no longer just a single memory.

It becomes part of a broader context around purchasing behaviour, personal preferences, and decision making.

This additional context fundamentally changes how memories are interpreted.

A vector embedding can capture semantic similarity between individual pieces of text.

A hierarchy captures something different.

It captures **relationships**.

Those relationships provide meaning that individual memories cannot represent on their own.

This richer representation benefits retrieval in multiple ways.

Hybrid retrieval techniques such as BM25 gain additional lexical context from the hierarchical path, allowing related concepts to reinforce one another even when they do not explicitly appear in the original memory.

Semantic retrieval also improves because related memories naturally cluster around common concepts instead of drifting into disconnected fragments over time.

Consider a memory containing only the sentence *"User prefers PyTorch."* In isolation, that memory shares very little lexical overlap with a broad query such as *"Tell me about my machine learning journey."* Once the memory is placed within the hierarchy, however, it implicitly inherits its semantic path—*Artificial Intelligence → Machine Learning → Deep Learning → PyTorch*. Whether that path is indexed directly or exposed as metadata during retrieval, lexical methods such as BM25 can now match not only the specific memory but also the broader concepts surrounding it. Rather than searching isolated pieces of text, retrieval benefits from the contextual vocabulary provided by the hierarchy, substantially improving candidate recall for broad, high-level queries.

The hierarchy therefore isn't an additional layer placed on top of memory.

It **becomes part of the memory itself**.

From Organization to Understanding

Initially, I believed organizing memories into a hierarchy would solve most retrieval problems.

It solved some of them.

It didn't solve broad reasoning.

Questions such as:

"Summarize my machine learning journey."

or

"Tell me everything we've discussed while building MindCache."

don't require a handful of memories.

They require reasoning over hundreds.

Even with a well-organized hierarchy, traversing every memory beneath a large topic quickly becomes impractical.

The challenge was no longer locating the correct branch of the hierarchy.

It was efficiently understanding everything contained within it.

That led to the next architectural evolution.

Instead of treating internal nodes as simple organizational categories, they became **abstractions** of the knowledge beneath them.

Inspired by RAPTOR, every internal topic maintains a summary of the information contained in its descendants.

Rather than retrieving hundreds of individual memories, retrieval can first navigate progressively more abstract summaries before drilling down only into the branches that are relevant.

The hierarchy therefore serves two purposes simultaneously.

It organizes knowledge.

It summarizes knowledge.

That distinction is subtle, but it dramatically improves the system's ability to answer broad, multi-topic questions while keeping retrieval computationally efficient.

A Hierarchy That Evolves

A static hierarchy quickly becomes outdated.

New conversations introduce concepts.

Existing projects evolve.

Preferences change.

Topics split into more specialized branches, while others naturally merge over time.

If summaries remain unchanged, the abstraction layer gradually diverges from the knowledge it represents.

Regenerating the entire hierarchy after every conversation would keep everything synchronized, but it would also become prohibitively expensive as the system grows.

MindCache instead treats summaries as **living artifacts**.

Each summary maintains explicit links to the information it represents.

When new conversations modify a branch of the hierarchy, only the affected summaries are regenerated, allowing the abstraction layer to evolve incrementally alongside the underlying memories.

The result is a hierarchy that continuously reorganizes and refines itself instead of remaining a static index.

By this point, the knowledge base had become far more than an organizational tool.

It had become the backbone of the entire memory system.

The next question, however, proved just as important.

Even if knowledge is perfectly organized, should every memory really be represented in the same way?

4.2 — Representing Different Kinds of Memory

Organizing memories into a hierarchy solved one important problem.

Every memory now had context.

Related concepts naturally clustered together, broad queries could traverse hierarchical summaries instead of hundreds of individual memories, and retrieval could reason over knowledge rather than isolated fragments.

But another limitation quickly became apparent.

The hierarchy assumed every memory behaved the same way.

In practice, they didn't.

A user preference, a factual observation, a project milestone, and an architectural decision may all originate from conversations, but they evolve very differently over time. They differ in how frequently they change, how often they should influence future reasoning, and how they should be maintained as new information arrives.

Treating them identically forced retrieval to make unnecessary compromises.

Instead of asking,

"What should be remembered?"

MindCache began asking a different question.

"What kind of memory is this?"

That distinction fundamentally changed the architecture.

Four Memory Types

Rather than storing every piece of information in a single representation, MindCache gradually evolved four specialized memory types.

User Memories

User memories capture relatively persistent information about the person interacting with the system.

These include:

- Preferences
- Long-term goals
- Habits
- Constraints
- Personal workflows

Unlike ordinary factual memories, user memories often influence conversations that appear completely unrelated.

For example, a user's preferred programming language, purchasing habits, or communication style may affect hundreds of future responses.

Because of their broad influence, user memories are treated as persistent behavioral context rather than isolated facts.

Knowledge Memories

Knowledge memories represent relatively stable information learned during conversations.

Examples include:

- Technical concepts
- Domain knowledge
- Project documentation
- Research findings
- Reference material

Unlike user memories, knowledge memories primarily answer questions directly rather than influencing how answers should be generated.

They naturally accumulate over time and benefit from retrieving multiple related memories simultaneously.

Episodic Memories

Not everything important is a fact.

Some information only makes sense as part of a sequence of events.

Episodic memories capture experiences such as:

- Project milestones
- Completed tasks
- Important conversations
- Failures
- Achievements

Unlike knowledge memories, episodic memories preserve chronology.

They allow the system to reconstruct journeys, understand progress over time, and answer questions such as:

"How did this project evolve?"

or

"What happened before we switched approaches?"

Their value lies not in individual events, but in the timeline they collectively create.

Decision Memories

Decision memories became the most distinctive component of the architecture.

Long-running projects rarely produce decisions in a single conversation.

Instead, they evolve gradually.

Ideas are proposed.

Alternatives are explored.

Trade-offs are discussed.

Earlier assumptions are revisited.

Eventually, a decision stabilizes.

Representing each discussion as an independent memory preserves the conversations but loses the reasoning that connects them.

MindCache instead models decisions as evolving entities.

Each decision maintains its current state together with the discussions, supporting evidence, and alternatives that contributed to it.

Rather than remembering only the final outcome—or forcing retrieval to reconstruct months of architectural discussions—the system preserves the evolution of the decision itself.

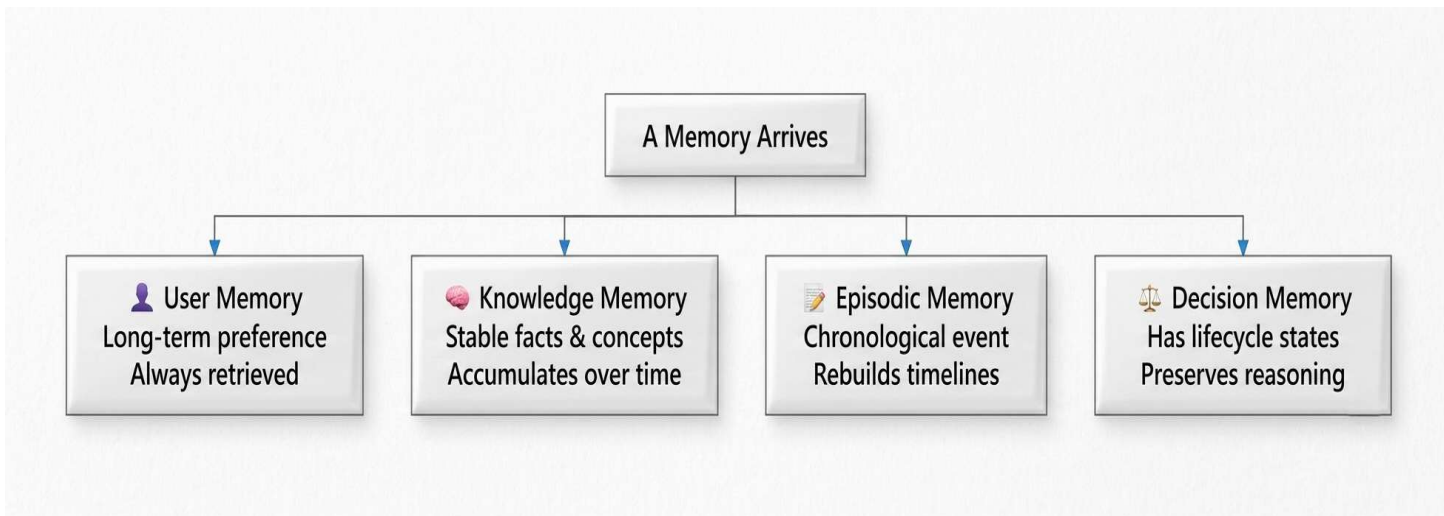


Figure 3. MindCache models different kinds of information using specialized memory representations. Each memory type follows its own lifecycle, update strategy, and retrieval behavior.

Decision Analysis

Supporting specialized memory types required more than simply assigning labels.

The system also needed to determine how memories should evolve over time.

This was particularly important for decision memories.

When a new conversation arrives, the system doesn't simply ask:

"Did we make a decision?"

Instead, it evaluates how that conversation relates to decisions that already exist.

Questions such as:

- Does this introduce a new decision?
- Does it refine an existing one?
- Has an earlier proposal been rejected?
- Has the decision reached a stable conclusion?

allow the system to maintain coherent decision histories instead of accumulating disconnected discussions.

This responsibility belongs to the **Decision Analyzer**, whose purpose is not merely to extract decisions, but to maintain their lifecycle as projects evolve across weeks or even months of conversation.

As a result, retrieval no longer has to reconstruct architectural reasoning from dozens of individual memories.

The reasoning process has already been preserved.

Beyond Classification

At first glance, memory types may appear to be little more than organizational categories.

In practice, they influence almost every component of the architecture.

Each memory type defines:

- how information is updated,
- how conflicting information is resolved,
- how aggressively memories are retrieved,
- how they contribute to summaries,
- and how they evolve as new conversations arrive.

The architecture is no longer storing generic memories.

It is modeling how different kinds of knowledge behave.

That distinction becomes especially important during ingestion.

Once memories can evolve independently, the system needs a mechanism to coordinate those changes across the hierarchy, summaries, decision histories, and graph structure.

That responsibility belongs to the final major component of MindCache.

Smart Ingestion..

4.3 — Smart Ingestion: Orchestrating an Evolving Knowledge System

By this point, the architecture consisted of several independent components.

The knowledge hierarchy organized information into semantic topics.

Hierarchical summaries enabled efficient reasoning over large branches of knowledge.

Specialized memory types modeled how different kinds of information evolved.
Decision memories preserved long-running reasoning instead of isolated conclusions.
Individually, each component solved a specific problem.
Collectively, they introduced a new one.
Every new conversation could potentially affect all of them.
A single interaction might introduce a new topic, update an existing user preference, continue an architectural decision, invalidate an earlier assumption, modify several summaries, and trigger changes throughout the knowledge hierarchy.
Maintaining these components independently would quickly become inconsistent.
The system needed a mechanism capable of coordinating every update while preserving the integrity of the knowledge base.
That mechanism became **Smart Ingestion**.
Rather than acting as a storage pipeline, Smart Ingestion serves as the orchestration layer responsible for continuously maintaining the entire memory system.

From Storage to Knowledge Evolution

Traditional memory pipelines are relatively straightforward.

Conversation



Extract Memories



Generate Embeddings



Store

Once the memories are stored, ingestion is complete.
MindCache follows a fundamentally different workflow.
Every conversation becomes an opportunity to refine everything the system already knows.
Instead of asking a single question—

"What should I remember?"

Smart Ingestion asks a sequence of increasingly richer questions.

Conversation



Extract candidate memories



Identify memory type



Find related memories



Determine whether this updates existing knowledge



Update decision histories if necessary



Assign or create hierarchy topics



Update affected summaries



Evaluate hierarchy health



Trigger maintenance when required



Persist changes

Notice where storage appears.
It is the final step.
Understanding happens first.

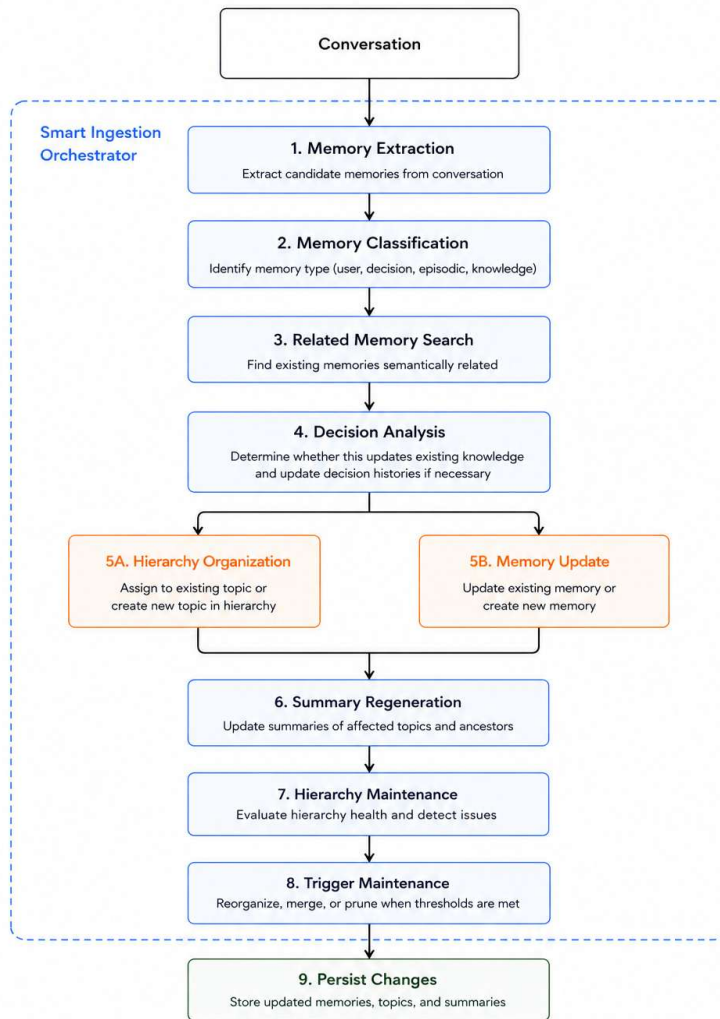


Figure 4. Smart Ingestion orchestrates the evolution of the knowledge base by coordinating memory extraction, classification, organization, summarization, and maintenance before persisting new information.

Maintaining the Knowledge Hierarchy

Unlike traditional databases, a knowledge hierarchy naturally degrades over time.

New concepts appear.

Projects expand into new domains.

Preferences evolve.

Some topics become overcrowded.

Others gradually become obsolete.

Simply inserting new memories eventually produces an increasingly fragmented knowledge graph.

Smart Ingestion therefore treats every conversation as a maintenance operation rather than a write operation.

Depending on the conversation, the correct action may be to:

- create a new topic,
- strengthen an existing one,
- merge semantically similar concepts,
- update an existing memory,
- extend a decision history,
- regenerate summaries,
- or make no structural changes at all.

Sometimes the most valuable outcome is not creating another memory.

It is improving one that already exists.

This shift fundamentally changes how the system grows.

The objective is no longer to accumulate memories.

It is to continuously improve the quality of the knowledge base.

Coordinating the Architecture

As MindCache evolved, Smart Ingestion gradually became responsible for every major subsystem.

It coordinates:

- memory extraction,
- memory classification,
- topic assignment,
- decision analysis,
- hierarchy maintenance,
- summary regeneration,
- metadata updates,
- and graph maintenance.

Each component depends on the others.

Updating a decision may require updating summaries.

Adding a new topic may affect the hierarchy.

Modifying the hierarchy may require reorganizing neighbouring concepts.

Without a central orchestration layer, these updates would quickly become inconsistent.

Smart Ingestion ensures that every architectural component evolves together.

Scaling Continuous Maintenance

As the hierarchy continued to grow, another practical challenge emerged.

Reorganizing the entire graph after every conversation became increasingly expensive.

Most updates affected only a small portion of the hierarchy, yet rebuilding the complete structure required processing thousands of unrelated nodes.

To avoid this, MindCache partitions the hierarchy into smaller semantic communities using the **Leiden community detection algorithm**.

Each community can then be reorganized independently before its local improvements are merged back into the global hierarchy.

This significantly reduces computational cost while preserving the overall organization of the knowledge graph.

Rather than repeatedly rebuilding the entire system, maintenance becomes localized to the areas that have actually changed.

From Memory Storage to Knowledge Evolution

Looking back, Smart Ingestion became much more than an ingestion pipeline.

It is the component responsible for ensuring that every conversation improves the quality of the knowledge base rather than simply increasing its size.

The hierarchy remains organized.

Summaries remain synchronized.

Decision histories continue to evolve.

Memory types preserve their individual lifecycles.

The graph continuously adapts as new information arrives.

The result is a memory system that doesn't merely accumulate conversations.

It continuously reorganizes, refines, and strengthens its understanding of them.

Only after building an evolving knowledge base does the final challenge begin.

How can that knowledge be retrieved efficiently, consistently, and within the strict latency requirements of an interactive AI system?

That question shaped the final component of MindCache.

Retrieval.

Part V — Retrieval: From Search to Evidence Assembly

By the time the architecture of MindCache had stabilized, the retrieval problem looked very different from the one I had started with.

The hierarchy organized knowledge.

Specialized memory types modeled different kinds of information.

Decision histories preserved long-running reasoning.

Smart Ingestion continuously maintained the knowledge base as conversations evolved.

The system already understood its memories.

Retrieval no longer had to create that understanding.

It only had to expose the right parts of it.

That distinction fundamentally changed the role of retrieval.

Traditional retrieval systems answer a simple question:

Which stored memories are most similar to this query?

MindCache asks a different one.

What evidence does the language model need to answer this question correctly?

That subtle change in objective shaped every retrieval decision that followed.

Retrieval Is an Evidence Problem

The simplest retrieval pipeline is familiar to anyone who has built a Retrieval-Augmented Generation (RAG) system.

Query



Embedding



Vector Search



Top-K Memories



Language Model

For many questions, this works remarkably well.

Specific factual questions naturally retrieve related memories.

Technical discussions retrieve relevant technical context.

Semantic search performs exactly as intended.

The limitation isn't that retrieval fails.

It's that similarity alone often isn't sufficient.

A broad summarization query requires representative evidence rather than the most similar memories.

A timeline question depends on chronology.

A decision question requires the current state of the decision together with the reasoning behind it.

A preference-oriented question may depend on a handful of user memories that are only weakly related to the wording of the query.

Each of these questions requires a different kind of evidence.

Retrieval therefore became an evidence assembly problem rather than a search problem.

Retrieval Begins with Intent

One of the most important observations during development was that the query itself often reveals how retrieval should behave.

Consider the following questions.

Which mattress should I buy?

Tell me about my machine learning journey.

Why did we decide to use SQLite?

Which books have I completed?

From a traditional retrieval perspective, these are all simply search queries.

From an evidence perspective, they are completely different.

The first depends primarily on long-term user preferences.

The second requires broad coverage across many topics.

The third needs a coherent decision history.

The fourth depends heavily on episodic memories arranged chronologically.

Rather than treating every query identically, MindCache first determines the type of evidence most likely to be useful.

Only then does retrieval begin.

Memory Types Guide Retrieval

The introduction of specialized memory types naturally changed retrieval.

User memories are small but broadly influential.

Knowledge memories usually benefit from retrieving many related concepts simultaneously.

Episodic memories prioritize chronology over similarity.

Decision memories retrieve the current state together with the reasoning that produced it.

The objective is no longer

Which memories are most similar?

Instead, retrieval first asks

Which memories should influence this answer?

That simple distinction dramatically reduces irrelevant context while improving the quality of evidence available to the language model.

A Multi-Stage Retrieval Pipeline

Once the retrieval objective shifted from search to evidence assembly, a single retrieval algorithm was no longer sufficient.

Instead, MindCache performs retrieval in several stages, each solving a different problem.

Stage 1 — Candidate Generation

The first stage focuses on recall.

No single retrieval method consistently retrieves every useful memory.

Vector search excels at semantic similarity but often misses exact terminology.

BM25 captures lexical relationships but struggles with paraphrases and conceptual similarity.

Rather than choosing between them, MindCache combines both approaches to generate a broader candidate set.

At this stage the objective is simple:

Don't miss useful evidence.

Stage 2 — Rank Fusion

Candidate generation produces multiple rankings.

Semantic similarity.

Lexical similarity.

Hierarchy.

Memory metadata.

Each captures a different notion of relevance.

MindCache combines these rankings using **Reciprocal Rank Fusion (RRF)**.

Instead of assuming one retriever is superior, RRF rewards memories that consistently appear across multiple retrieval methods.

The result is a more stable candidate ranking without introducing significant computational cost.

Stage 3 — Adaptive Retrieval

Not every question requires the same retrieval strategy.

Specific factual questions usually retrieve a small number of detailed memories.

Broad questions first traverse hierarchical summaries before expanding into detailed evidence.

Decision-oriented queries prioritize Decision Memories.

Preference-oriented queries emphasize User Memories.

Timeline reconstruction relies primarily on Episodic Memories.

Retrieval therefore adapts its strategy according to the evidence likely to be required rather than applying a fixed pipeline to every query.

Stage 4 — Context Assembly

The final stage is not simply selecting the highest-ranked memories.

Its objective is to assemble the smallest collection of evidence that enables the language model to answer the user's question correctly.

Sometimes that consists of a handful of user memories.

Sometimes it requires summaries together with detailed evidence.

Sometimes it combines multiple memory types into a single coherent context.

The emphasis shifts from maximizing similarity to maximizing reasoning quality.

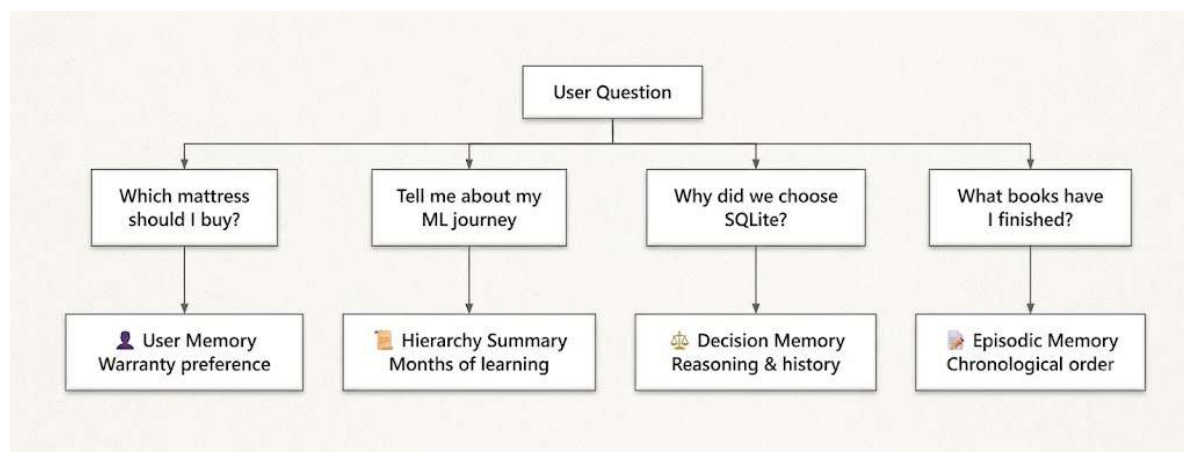


Figure 5. The final retrieval pipeline assembles evidence through multiple complementary stages, progressively refining the candidate set before constructing the context passed to the language model.

Retrieval Works Because Ingestion Already Did the Hard Work

One of the biggest lessons from building MindCache is that retrieval quality depends surprisingly little on retrieval itself. By the time a query reaches the retrieval pipeline, most of the difficult work has already been completed. The hierarchy has organized the knowledge. Smart Ingestion has integrated new conversations. Memory types have separated different kinds of information. Decision histories preserve long-running reasoning. Hierarchical summaries provide abstractions over large branches of knowledge. Retrieval simply takes advantage of that preparation. Looking back, this completely changed how I think about long-term memory systems. At the beginning of the project, I believed retrieval was responsible for discovering intelligence within the memory store. By the end, I realized something quite different.

Retrieval shouldn't create understanding.

It should exploit understanding that has already been created.

That became one of the central architectural philosophies behind MindCache.

Rather than building an increasingly intelligent retrieval pipeline, the system invests its effort in preparing knowledge during ingestion so retrieval can remain lightweight, predictable, and latency efficient.

The final question, then, was whether that philosophy actually worked.

The only way to answer it was through evaluation.

The implementation described throughout this article is available as the open-source **MindCache** library on GitHub, together with installation instructions and examples.

- **GitHub:** [faisalhussain-devs/MindCache](https://github.com/faisalhussain-devs/MindCache)
- **PyPI:** `pip install mindcache`

Part VI — Evaluation: Measuring an Evolving Memory System

Building an architecture is only half the challenge.

The more important question is whether that architecture actually improves long-term memory.

Throughout the development of MindCache, evaluation was never treated as a final validation step. Every benchmark exposed new failure modes, and each failure was traced back to the component responsible—whether ingestion, memory organization, retrieval, or reasoning. Those observations directly shaped the architecture described throughout this article.

Rather than asking *"How accurate is the system?"*, the evaluation process asked a more useful question:

What does each failure reveal about the architecture?

That mindset turned benchmarking into another engineering feedback loop rather than a final scorecard.

Evaluating on BEAM

The final architecture was evaluated using **BEAM**, an ICLR 2026 benchmark designed specifically for long-term conversational memory. Unlike traditional retrieval benchmarks, BEAM evaluates abilities that closely resemble real-world assistants operating over months of conversation. Questions require systems to reason across evolving user preferences, temporal relationships, contradictory information, knowledge updates, multi-session conversations, and broad summaries that span many topics.

These are precisely the challenges that motivated MindCache's architecture.

For the final evaluation, MindCache was tested across six conversations—four from the **BEAM-1M** dataset and two from **BEAM-10M**—covering a total of **140 evaluation questions**.

Benchmark	Questions	Correct	Accuracy
BEAM-1M	80	73	91.25%
BEAM-10M	40	30	75.0%
Overall	120	103	85.8%

Table 2. BEAM Benchmark Results

The difference between the two datasets is unsurprising.

While the 1M conversations already require substantial long-term reasoning, the 10M benchmark introduces significantly larger conversational histories together with evolving information, contradictory statements, and much broader retrieval requirements. Maintaining strong performance under those conditions was one of the primary design goals of MindCache.

Retrieval Performance

Accuracy was only one objective.

A memory system must also remain fast enough to support interactive conversations.

Earlier versions of MindCache relied on cross-encoder reranking to improve retrieval quality. While effective, reranking quickly became the dominant source of latency, particularly for broad queries with large candidate sets. The final architecture instead relies on hybrid retrieval, memory-aware ranking, and evidence planning to produce high-quality context without requiring expensive reranking. This reduced average retrieval latency from approximately **25 seconds** to **1.08 seconds**—an improvement of more than **23×**—while maintaining competitive retrieval quality. That result reinforced one of the central design principles behind MindCache:

Preparation during ingestion is often more valuable than increasingly complex retrieval.

Understanding the Remaining Failures

Perhaps the most valuable outcome of the evaluation was not the benchmark score itself, but understanding why the remaining failures occurred. Rather than being distributed evenly across different capabilities, they clustered into a surprisingly small number of categories.

Failure Category	Approximate Share
Event ordering & temporal reasoning	~70%
Broad summarization	~20%
Knowledge updates	~10%

Table 3. Remaining Failure Categories

This distribution is significant because it shows that traditional retrieval is no longer the dominant bottleneck. Most factual, preference-based, and decision-oriented questions were handled reliably. The remaining challenges emerged primarily when the system needed to assemble evidence across many different branches of the knowledge hierarchy while preserving temporal relationships between events. Unlike factual retrieval, these questions rarely have a single "correct" memory. Success depends on retrieving a sufficiently diverse collection of evidence while maintaining coherent coverage and chronology. That remains one of the hardest problems in long-term conversational memory.

What the Evaluation Validated

The benchmark results also provided evidence that the architectural decisions described throughout this article were addressing the intended problems. Hierarchical organization enabled broad navigation across large knowledge bases without exhaustively searching every memory. Specialized memory types allowed retrieval to prioritize different forms of evidence depending on the question being asked. Smart Ingestion ensured that the knowledge base evolved continuously rather than becoming an ever-growing collection of disconnected memories. Hybrid retrieval combined semantic and lexical signals to improve candidate recall while remaining computationally efficient. Taken together, these components produced a system capable of maintaining high retrieval accuracy while scaling to conversations containing millions of tokens. More importantly, the evaluation demonstrated that long-term memory quality depends on far more than retrieval algorithms alone. The quality of the retrieved evidence is largely determined before retrieval even begins.

The evaluation demonstrated that MindCache performs reliably across a wide range of long-term memory tasks while also revealing the next generation of challenges. Broad evidence assembly, temporal reasoning, and multi-topic synthesis remain difficult problems—not only for MindCache, but for long-term conversational memory systems in general. Those observations directly shape the future directions discussed in the next chapter.

Part VII — Reflections: What Building MindCache Taught Me

When I started this project, I thought I was building a better memory system. Six months later, I don't think that's what I built. More importantly, I don't think that's what the real problem is anymore. Almost every major architectural decision in MindCache came from discovering that one of my earlier assumptions was wrong. The system rarely improved because I found a better algorithm. It improved because I gradually stopped asking the wrong questions. Looking back, there are a handful of lessons that fundamentally changed how I think about long-term memory.

Retrieval Was Never the Hardest Problem

The biggest surprise of the project was also the simplest.

When I began, I assumed retrieval was where most of the intelligence should live. Whenever the system failed, my instinct was to improve retrieval—better embeddings, better rerankers, better ranking algorithms. That approach worked for a while. Eventually it stopped. Retrieval operates under strict latency constraints. By the time a user asks a question, the system has only a short window to decide what information matters. It cannot afford to reorganize a knowledge graph, reinterpret months of conversation, or rebuild a user's history while they wait for a response. Ingestion has none of those constraints. It happens before the question is asked. That makes it the ideal place to spend computational effort understanding conversations, organizing knowledge, updating decisions, and maintaining the structure of the memory system. By the time retrieval begins, most of the difficult work should already be finished. This single realization influenced almost every component of MindCache.

Memory Is More Than Storage

Another assumption quietly disappeared during development. Initially, I viewed memory as little more than persistent storage. Extract information. Generate embeddings. Store it. Retrieve it later. The longer I worked on the project, the less convincing that model became. Knowledge changes. Projects evolve. Preferences shift. Old assumptions become obsolete. New experiences reshape earlier decisions. A useful memory system cannot simply accumulate more information. It must continuously improve what it already knows. That means reorganizing knowledge, updating relationships, maintaining summaries, resolving contradictions, and refining existing memories as conversations evolve. Looking back, the most important components of MindCache are not the ones that store memories. They are the ones that help those memories evolve.

Different Memories Live Different Lives

One of the biggest architectural changes came from realizing that memories are not all the same. Facts accumulate. Experiences form timelines. Preferences evolve gradually. Decisions move through stages before eventually stabilizing—or being replaced entirely. Treating all of them as identical pieces of text made both storage and retrieval unnecessarily complicated. Separating them into different memory types simplified the architecture, but more importantly, it made the system behave more like the information it was trying to represent. The goal was no longer to store conversations. It was to model how knowledge itself changes over time.

The Next Challenge Isn't Retrieval

The evaluation highlighted something I hadn't expected. Most remaining failures were no longer caused by missing individual memories. They appeared when the system needed to assemble evidence scattered across many different parts of the knowledge base. Questions such as *"Summarize my machine learning journey."* or *"How has MindCache evolved over the last six months?"* rarely depend on one important memory. They require balancing breadth, chronology, and relevance across dozens—or sometimes hundreds—of interconnected memories. Finding individual evidence has become increasingly reliable. Understanding how to combine that evidence into coherent reasoning remains a much harder problem. In my opinion, this is one of the most important open challenges in long-term conversational memory.

Looking Beyond MindCache

Building the first version of MindCache answered many questions.

It also raised even better ones.

How should a memory system decide which information deserves to survive for years?

Can memories reorganize themselves without explicit maintenance?

Should rarely used memories gradually disappear, or simply become less prominent?

Can multiple AI agents develop a shared memory while preserving their own individual experiences?

How can knowledge continue evolving without making retrieval progressively slower?

These questions extend beyond the architecture presented in this article.

They describe the problems I find most exciting to explore next.

A Different Way to Think About Memory

One idea kept resurfacing throughout the project, but never became part of the final architecture.

Today, nearly every memory system—including MindCache—makes essentially the same decision during ingestion.

Should this information be stored?

After months of building and evaluating the system, I began to wonder whether that was the wrong question.

Some memories remain untouched for months before suddenly becoming the single most important piece of information for answering a user's question.

A simple preference such as

"Whenever I buy something expensive, warranty matters more than anything else."

may be irrelevant for hundreds of conversations before completely determining the correct answer to a product recommendation.

Other memories might be retrieved frequently while contributing very little to the final reasoning process.

Traditional retrieval systems often treat both memories similarly because they measure relevance only after a query arrives.

What if memory systems could reason about something different?

Not similarity.

Utility.

Instead of asking,

"How similar is this memory to the current query?"

a future memory system might ask,

"How much will this memory improve the quality of future reasoning?"

That is a much more difficult problem.

It requires estimating the long-term influence of a memory before knowing the questions it will eventually help answer.

I believe this is one of the most promising directions for future research.

A mature memory system shouldn't merely retrieve relevant memories.

It should understand which memories are disproportionately valuable, preserve them more carefully, surface them more aggressively when needed, and perhaps even reorganize itself around them over time.

Final Thoughts

When this project began, I simply wanted ChatGPT to remember things about me.

I thought I was building a personal memory extension.

Instead, it became an exploration of how intelligent systems should organize knowledge throughout their lifetime.

Many of the individual ideas behind MindCache are not new. Hierarchical organization, graph-based representations, recursive summarization, and hybrid retrieval have all appeared throughout the literature.

The contribution of MindCache was never to invent each of those ideas independently.

It was to explore how they could work together as a single, continuously evolving memory system.

If there is one lesson I hope readers take away from this project, it is this:

Long-term memory is not fundamentally a retrieval problem.

Retrieval is only the final step.

The real challenge begins much earlier.

It begins with deciding what deserves to be remembered, how that knowledge should be represented, how it should evolve as new information arrives, and how the system can continuously improve instead of simply accumulating more data.

After six months of building MindCache, I no longer think the most interesting question is,

"How can we help AI remember more?"

I think the more important question is,

"How should an AI remember?"

I don't believe MindCache is the final answer to that question.

But I hope it contributes, in some small way, to the conversation.

If you'd like to explore the implementation, experiment with the architecture, or contribute to the project:

- **GitHub:** [faisalhussain-devs/MindCache](https://github.com/faisalhussain-devs/MindCache)
- **PyPI:** pip install mindcache

References

1. BEAM: *Benchmarking Long-Term Memory in LLM Agents* (ICLR 2026)
2. RAPTOR: *Recursive Abstractive Processing for Tree-Organized Retrieval*
3. Mem0: *Building Production-Ready Memory for AI Assistants*
4. MemGPT: *Towards LLMs as Operating Systems*
5. Zep Documentation
6. LangMem Documentation